

Langevin-algorithms-for-deep-stochastic-control

Implementation of the paper Langevin algorithms for Markovian Neural Networks and Deep Stochastic control in pytorch.

Inspired by the work of Pierre Bras, Ph.D.

Framework

Stochastic Optimal Control

We use the regular continuous-time finite-horizon SOC framework and notations, so that our optimization problem reads:

$$\min_u J(u) := \mathbb{E} \left[\int_0^T G(t, X_t) dt + F(X_T) \right]$$

subject to the stochastic differential equation:

$$\begin{cases} dX_t = b(X_t, u_t) dt + \sigma(X_t, u_t) dW_t, & t \in [0, T] \\ X_0 = x_0 \end{cases}$$

where $b : \mathbb{R}^{d_1} \times \mathbb{R}^{d_3} \rightarrow \mathbb{R}^{d_1}$, $\sigma : \mathbb{R}^{d_1} \times \mathbb{R}^{d_3} \rightarrow \mathcal{M}_{d_1, d_2}(\mathbb{R})$, W is a $\mathbb{R}^{d_2}$ -valued Brownian motion and u is a $\mathbb{R}^{d_3}$ -valued continuous adapted process, $T > 0$, $G : [0, T] \times \mathbb{R}^{d_1} \rightarrow \mathbb{R}$ and $F : \mathbb{R}^{d_1} \rightarrow \mathbb{R}$.

The process $(X_t)_{t \in [0, T]}$ is approximated through the Euler-Maruyama scheme, and likewise the control u_t is replaced by its discrete-time counterpart. We consider the regular subdivision of $[0, T]$:

$$t_k := kT/N, \quad k \in \{0, \dots, N\}, \quad h := T/N$$

and we approximate the control applied at times $t_0, \dots, t_{N-1}$ either as the output of a single neural network depending on t , or as the output of N neural networks, one for each discretization instant t_k :

$$u_{t_k} = \bar{u}_\theta(t_k, X_{t_k}) \quad \text{or} \quad u_{t_k} = \bar{u}_{\theta^k}(X_{t_k})$$

where $\bar{u}_\theta$ is a neural function with finite-dimensional parameter $\theta \in \mathbb{R}^d$. Indeed, since $(X_t)_{t \in [0, T]}$ a Markovian process, we can assume that u_t depends only on t and on X_t instead of t and $(X_s)_{s \in [0, t]}$.

The SOC problem is thus approximated by

$$\min_\theta \bar{J}(\bar{u}_\theta) := \mathbb{E} \left[\sum_{k=0}^{N-1} (t_{k+1} - t_k) G(t_{k+1}, \bar{X}_{t_{k+1}}^\theta) + F(\bar{X}_{t_N}^\theta) \right]$$

$$\text{s.t.} \quad \begin{cases} \bar{X}_{t_{k+1}}^\theta = \bar{X}_{t_k}^\theta + (t_{k+1} - t_k) b(\bar{X}_{t_k}^\theta, \bar{u}_{k,\theta}(\bar{X}_{t_k}^\theta)) + \sqrt{t_{k+1} - t_k} \sigma(\bar{X}_{t_k}^\theta, \bar{u}_{k,\theta}(\bar{X}_{t_k}^\theta)) \xi_{k+1} \\ \xi_k \sim \mathcal{N}(0, I_{d_2}) \\ \bar{X}_{t_0}^\theta = x_0 \end{cases}$$

The SGD algorithm reads

$$\theta_{n+1} = \theta_n - \gamma_{n+1} \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} \nabla_{\theta} \bar{J}(\bar{u}_{\theta_n}, (\xi_k^{i,n+1})_{1 \leq k \leq N}) =: \theta_n - \gamma_{n+1} g_{n+1}$$

where $(\xi_k^{i,n})_{1 \leq k \leq N, 1 \leq i \leq n_{\text{batch}}, n \in \mathbb{N}}$ is an array of i.i.d. $\mathcal{N}(0, I_{d_2})$ -distributed random vectors, $(\gamma_n)_{n \in \mathbb{N}}$ is a nonincreasing positive step sequence and where the dependence of $\bar{J}$ in $(\xi_k^{i,n})$ is made explicit.

Adaptive Langevin optimizers

We use the popular preconditioned variations of SGD Adam 1, RMSprop 2, and Adadelta 3. We also study an adaptively preconditionned version of SGD with momentum introduced in 4 and a Langevin adaptation of Adagrad. Just like in the original paper, the Langevin version of an algorithm is *L - name*, the gradient update thus reads:

$$\theta_{n+1} = \theta_n - \gamma_{n+1} P_{n+1} \cdot g_{n+1} + \sigma_{n+1} \sqrt{\gamma_{n+1}} \mathcal{N}(0, P_{n+1})$$

Where $(P_n)_n$ is the preconditioner, $(g_n)_n$ is the gradient estimate as defined below, $(\gamma_n)_n$ is the learning rate schedule and $(\sigma_n)_n$ is the noise scale schedule.

We also test the $p\%$ Layer Langevin algorithm introduced in 5, which injects noise only to the first $p\%$ layers of the network.

Numerical setting and implementation

We reproduce all the experiments displayed in 6, while testing additional settings in an attempt to be as thorough as possible. We kept the same batch size of $n_{\text{batch}} = 512$. Each epoch is trained on 5 batches and evaluated on 25 batches. We also plot the corresponding 95% confidence intervals for $J(u_\theta)$ even when too small to be perceived. For our results to make sense, we made sure to initialize the weights to the exact same values when comparing different algorithms. A **torch.Generator** instance is created, using a seed that can be passed as an argument in the control configuration, during the call of the network class. We initialized all weights from a $\mathcal{N}(0, 0.1)$ just like in the paper, except for the w parameter in the deephedging problem which was initialized to zero. Other more sophisticated initializations, like the Xavier initialization, are possible with our implementation but were not tested.

Algorithm 1 Adam

Parameters: $\beta_1, \beta_2, \lambda > 0$

$$\begin{aligned} M_{n+1} &= \beta_1 M_n + (1 - \beta_1) g_{n+1} \\ \text{MS}_{n+1} &= \beta_2 \text{MS}_n + (1 - \beta_2) g_{n+1} \odot g_{n+1} \\ \widehat{M}_{n+1} &= M_{n+1} / (1 - \beta_1^{n+1}) \\ \widehat{\text{MS}}_{n+1} &= \text{MS}_{n+1} / (1 - \beta_2^{n+1}) \\ P_{n+1} &= \text{diag} \left(\mathbb{1} \oslash \left(\lambda \mathbb{1} + \sqrt{\widehat{\text{MS}}_{n+1}} \right) \right) \\ \theta_{n+1} &= \theta_n - \gamma_{n+1} P_{n+1} \cdot \widehat{M}_{n+1} \end{aligned}$$

Algorithm 3 Adadelta

Parameters: $\beta_1, \beta_2, \lambda > 0$

$$\begin{aligned} \text{MS}_{n+1} &= \beta_1 \text{MS}_n + (1 - \beta_1) g_{n+1} \odot g_{n+1} \\ P_{n+1} &= \text{diag} \left(\sqrt{\lambda \mathbb{1} + \widehat{\text{MS}}_n} \odot \sqrt{\lambda \mathbb{1} + \text{MS}_{n+1}} \right) \\ \theta_{n+1} &= \theta_n - \gamma_{n+1} P_{n+1} \cdot g_{n+1} \\ \widehat{\text{MS}}_{n+1} &= \beta_2 \widehat{\text{MS}}_n + (1 - \beta_2) (\theta_{n+1} - \theta_n) \odot (\theta_{n+1} - \theta_n) \end{aligned}$$

Algorithm 2 Adaptively Preconditioned SGLD

Parameters: $\Theta_0, \eta, \rho, \psi$

$$\begin{aligned} \mu_0 &= 0, \sigma_0 = 0 \\ \text{for } t = 1 \text{ to } T \text{ do} \\ & \widehat{g}_s(\Theta_t) = \nabla_{\Theta} \widehat{\mathcal{L}}_s(\Theta_t) \\ & \mu_t = \rho \mu_{t-1} + (1 - \rho) \widehat{g}_s(\Theta_t) \\ & C_t = \rho C_{t-1} + (1 - \rho) (\widehat{g}_s(\Theta_t) - \mu_t) (\widehat{g}_s(\Theta_t) - \mu_{t-1}) \\ & \xi_t \sim \mathcal{N}(\mu_t, C_t) \\ & \Theta_{t+1} = \Theta_t - \eta (\widehat{g}_s(\Theta_t) + \psi \xi_t) \\ \text{end for} \end{aligned}$$

Algorithm 4 RMSprop

Parameters: $\alpha, \lambda > 0$

$$\begin{aligned} \text{MS}_{n+1} &= \alpha \text{MS}_n + (1 - \alpha) g_{n+1} \odot g_{n+1} \\ P_{n+1} &= \text{diag} \left(\mathbb{1} \oslash (\lambda \mathbb{1} + \sqrt{\text{MS}_{n+1}}) \right) \\ \theta_{n+1} &= \theta_n - \gamma_{n+1} P_{n+1} \cdot g_{n+1} \end{aligned}$$

Algorithm 5 Adagrad

Parameters: $\gamma, \theta_0, f(\theta), \lambda, \tau, \eta$

$$\begin{aligned} \text{state_sum}_0 &= 0 \\ \text{for } t = 1 \text{ to } T \text{ do} \\ & g_t = \nabla_{\theta} f_t(\theta_{t-1}) \\ & \tilde{\gamma} = \gamma / (1 + (t-1)\eta) \\ & \text{if } \lambda \neq 0 \text{ then} \\ & \quad g_t = g_t + \lambda \theta_{t-1} \\ & \text{end if} \\ & \text{state_sum}_t = \text{state_sum}_{t-1} + g_t^2 \\ & \theta_t = \theta_{t-1} - \tilde{\gamma} \frac{g_t}{\sqrt{\text{state_sum}_t + \epsilon}} \\ \text{end for} \end{aligned}$$

Figure 1: Algorithms

The original paper already has an implementation in **tensorflow** but we chose to implement our optimizers in **pytorch** as we are more familiar with this framework. We built upon the official **pytorch** implementation of the different optimization algorithms. For the SOC problems, we went with a similar structure to that of the original repository. An SOC problem has to be a child class of **DeepControlledDiffusion**. This base class implements many abstract methods, the most important being **objective**, that takes as an argument an integer **batch_size** and returns a **torch.Tensor** containing the values of $J(u_\theta)$ for the different trajectories. We then use this tensor for the backward pass, and increment the weights of our networks using our optimizers. We also had to implement a custom learning rate scheduler that acts on the σ parameter. We attempted our numerical experiments using the same hyperparameters as the original papers, except in some cases where we found that the suggested values of σ were too high, leading to instability and divergence of our algorithms.

Stochastic optimal control

Fishing quotas

In this problem, we model the fish biomass $X_t \in \mathbb{R}^{d_1}$ by the following diffusion :

$$dX_t = X_t \odot ((r - u_t - \kappa X_t) dt + \eta dW_t), \quad t \in [0, T]$$

Where $u_t \in \mathbb{R}^{d_1}$ represents the control, and is constrained within $[u_m, u_M]^{d_1}$. We wish to keep the biomass close to a target value $\mathcal{X}_t \in \mathbb{R}^{d_1}$. The objective is defined as follows :

$$J(u) := \mathbb{E} \left[\int_0^T (|X_t - \mathcal{X}_t|^2 - \langle \alpha, u_t \rangle) dt + \beta [u]^{0,T} \right]$$

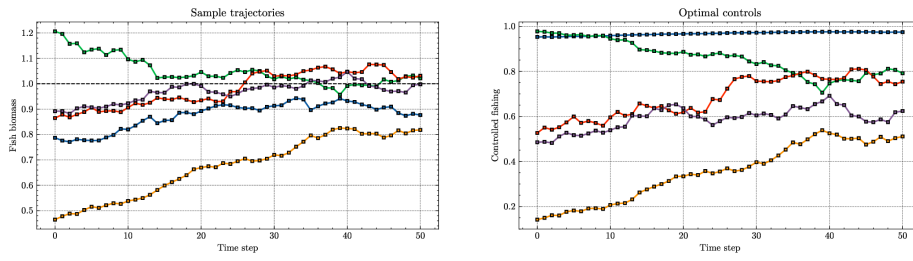
The quantity $[u]^{0,T}$ is the quadratic variation of the u and is approximated via $[u]^{0,T} \approx \sum_{k=0}^{N-1} |u_{t_{k+1}} - u_{t_k}|^2$. We worked in the same setting as the original paper:

$$d_1 = d_2 = 5, \quad T = 1, \quad \mathcal{X} = 1_{\mathbb{R}^5}, \quad r = 2 \times 1_{\mathbb{R}^5}, \quad \eta = 0.1 \times I_5, \quad \alpha = 0.01 \times 1_{\mathbb{R}^5}, \quad \beta = 0.1, \quad u_m = 0.1, \quad u_M = 1.$$

and :

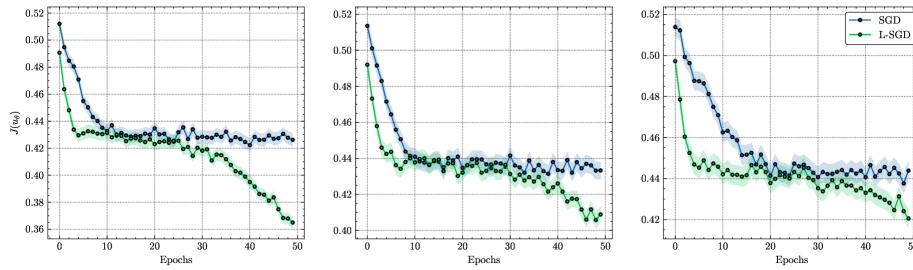
$$\kappa = \begin{bmatrix} 1.2 & -0.1 & 0 & 0 & -0.1 \\ 0.2 & 1.2 & 0 & 0 & -0.1 \\ 0 & 0.2 & 1.2 & -0.1 & 0 \\ 0 & 0 & 0.1 & 1.2 & 0 \\ 0.1 & 0.1 & 0 & 0 & 1.2 \end{bmatrix}$$

For more details on the problem, see 7.



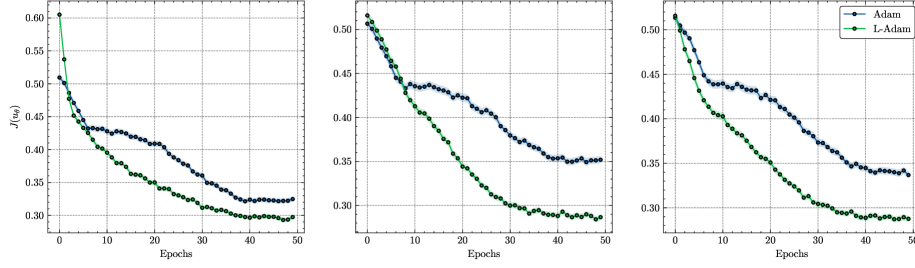
Example of a sample controlled trajectory for $N = 50$, on the left we see the components of $X_t \in \mathbb{R}^5$ and on the right $u_t \in \mathbb{R}^5$. The objective biomass $\mathcal{X}_t = 1_{\mathbb{R}^5}$ is drawn using a dotted horizontal black line. The initial state X_0 is sampled from $\mathcal{N}(1_{\mathbb{R}^5}, 0.5I_5)$ that is then clipped to $[0.2, 2]^5$.

Single control We first train a single control $u_\theta : \mathbb{R}_+ \times \mathbb{R}^5 \rightarrow \mathbb{R}^5$ given by a two-layer feedforward network, each layer containing 32 hidden units and activated by a ReLU function. The output is passed through a sigmoid function to ensure constraints on u . We train each algorithm for $N = 30, 50, 100$ and plot the test loss for the vanilla algorithm against the Langevin version.

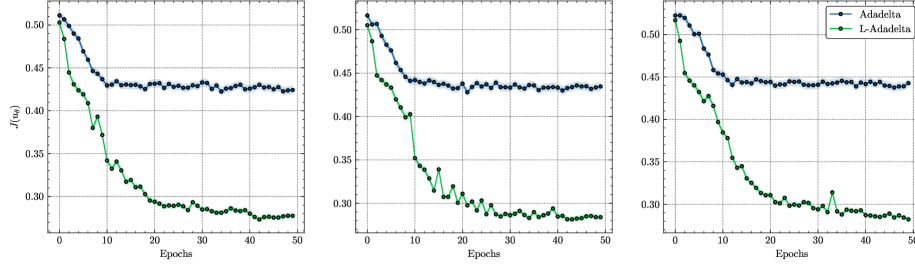


Comparison of SGD and L-SGD with momentum = 0.9 for $N = 30, 50, 100$ over 50 epochs. The schedules are $\gamma_n = 5e - 2$ and $\sigma_n = 1e - 3$ between epochs 0 and 40, after which γ_n is divided by 10 and σ set to 0.

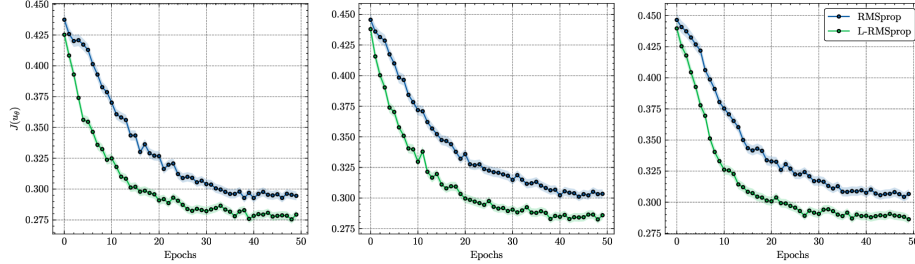
This example showcases the exploration capacity of Langevin dynamics, as we see both algorithms trapped in a local minimum. Thanks to the injected noise, the Langevin algorithm navigates the loss landscape better and escapes this point. Improvements are even more significant in the following algorithms.



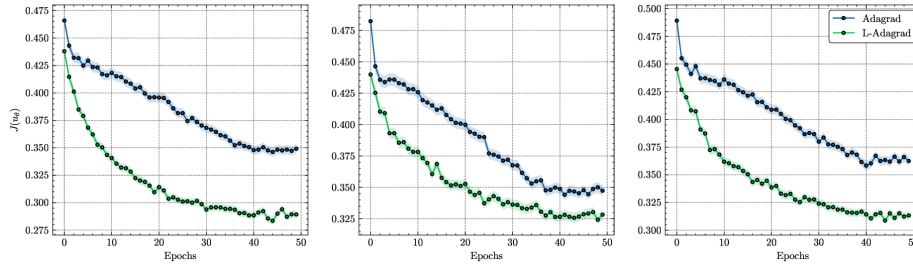
Comparison of Adam and L-Adam for $N = 30, 50, 100$ over 50 epochs. The schedules are $\gamma_n = 2e - 3$ and $\sigma_n = 1e - 3$ between epochs 0 and 40, after which γ_n is divided by 10 and σ set to 0.



Comparison of Adadelta and L-Adadelta for $N = 30, 50, 100$ over 50 epochs. The schedules are $\gamma_n = 5e - 1$ and $\sigma_n = 1e - 2$ between epochs 0 and 40, after which γ_n is divided by 10 and σ set to 0.



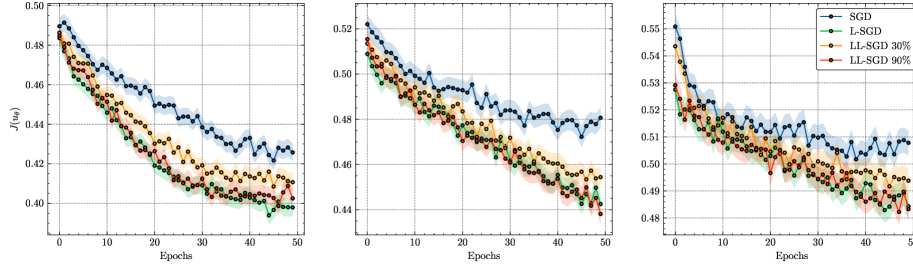
Comparison of RMSprop and L-RMSprop for $N = 30, 50, 100$ over 50 epochs. The schedules are $\gamma_n = 2e - 3$ and $\sigma_n = 5e - 3$ between epochs 0 and 40, after which γ_n is divided by 10 and σ set to 0.



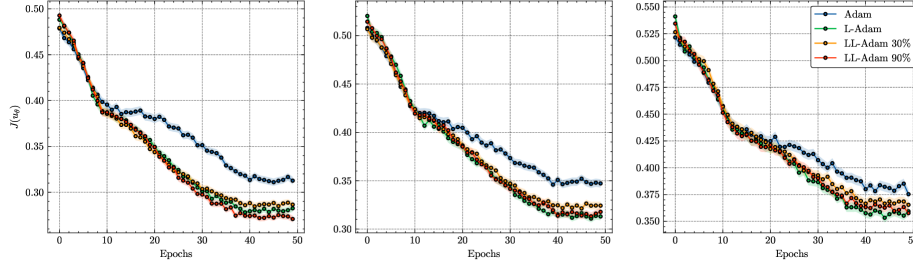
Comparison of Adagrad and L-Adagrad for $N = 30, 50, 100$ over 50 epochs. The schedules are $\gamma_n = 1e - 2$ and $\sigma_n = 5e - 5$ between epochs 0 and 40, after which γ_n is divided by 10 and σ set to 0.

Clearly, Langevin algorithms converge faster than their vanilla counterparts for the fishing quotas problem. More significant gains are also seen in the case of RMSprop compared to the original paper.

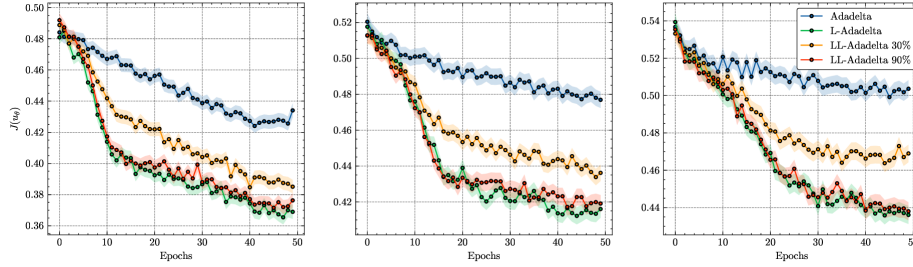
Multiple controls We now train multiple controls $(u_{\theta_k})_{0 \leq k \leq N}$, one for each discretization step. We compare all algorithms with their Langevin and $p\%$ Layer Langevin versions for $p = 30, 90$ and $N = 10, 20, 40$, respectively.



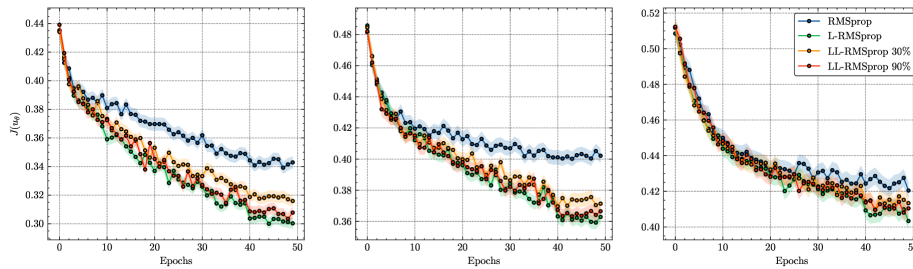
Comparison of SGD, L-SGD, and LL-SGD with momentum = 0.9 for $N = 10, 20, 40$ over 50 epochs. The schedules are $\gamma_n = 5e - 2$ and $\sigma_n = 2e - 3$ between epochs 0 and 40, after which γ_n is divided by 10 and σ set to 0.



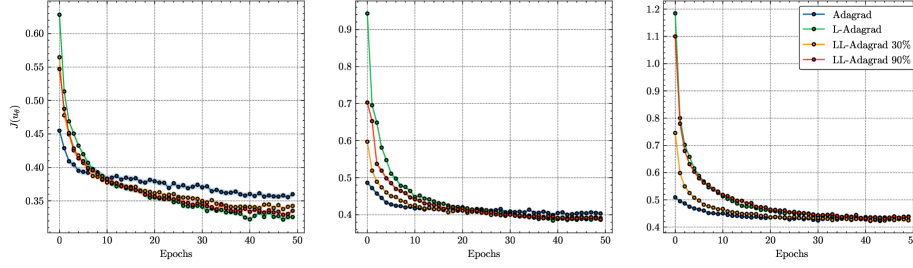
Comparison of Adam, L-Adam, and LL-Adam for $N = 10, 20, 40$ over 50 epochs. The schedules are $\gamma_n = 2e - 3$ and $\sigma_n = 2e - 4$ between epochs 0 and 40, after which γ_n is divided by 10 and σ set to 0.



Comparison of Adadelta, L-Adadelta, and LL-Adadelta for $N = 10, 20, 40$ over 50 epochs. The schedules are $\gamma_n = 5e - 1$ and $\sigma_n = 5e - 3$ between epochs 0 and 40, after which γ_n is divided by 10 and σ set to 0.



Comparison of RMSprop, L-RMSprop, and LL-RMSprop for $N = 10, 20, 40$ over 50 epochs. The schedules are $\gamma_n = 2e - 3$ and $\sigma_n = 2e - 3$ between epochs 0 and 40, after which γ_n is divided by 10 and σ set to 0.



Comparison of Adagrad, L-Adagrad, and LL-Adagrad for
 $N = 10, 20, 40$ **over 50 epochs.** The schedules are $\gamma_n = 1e - 2$ and
 $\sigma_n = 5e - 5$ between epochs 0 and 40, after which γ_n is divided by 10 and σ
set to 0.

As seen in the fishing quotas problem, the Langevin Layer algorithm does not improve significantly over regular Langevin algorithms, with the performance only improving when increasing the percentage of noisy layers.

Deep hedging

In this problem, we aim to replicate a $\mathcal{F}_T$ -measurable payoff Z on a portfolio $S_t \in \mathbb{R}^{d_1}$. The control $u_t \in \mathbb{R}^{d_1}$ represents the amount held at time t of each asset. The objective is:

$$J(u, w) := \mathbb{E} \left[w + l \left(Z - \sum_{k=0}^{N-1} \langle u_{t_k}, S_{t_{k+1}} - S_{t_k} \rangle + \sum_{k=0}^{N-1} \langle c_{t_k} S_{t_k}, |u_{t_k} - u_{t_{k-1}}| \rangle - w \right) \right]$$

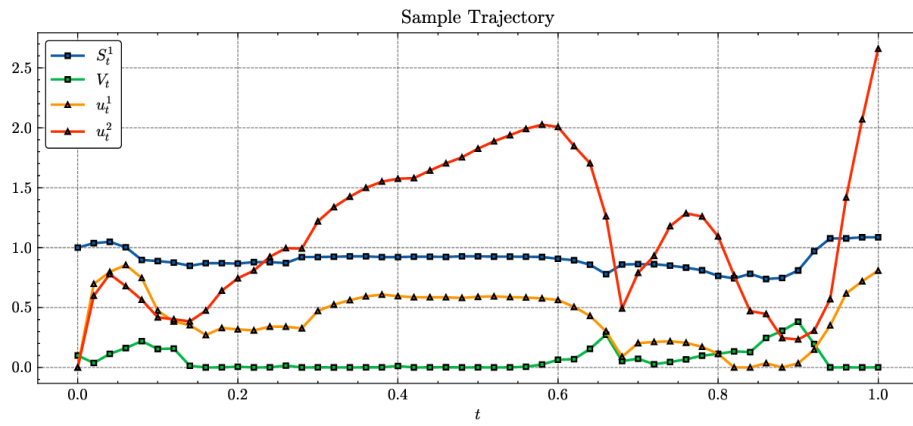
where $l : \mathbb{R} \rightarrow \mathbb{R}$ is a continuous, non-decreasing convex function. In our experiments, we consider the loss function associated with the value-at-risk, i.e., $l : x \mapsto (1 - \alpha)^{-1} \max(x, 0)$. We assume d_1 is even, such that for $d'_1 = d_1/2$, the assets $(S_t^{1,i})_{1 \leq i \leq d'_1} = (S_t^i)_{1 \leq i \leq d'_1}$ follow d'_1 independent Heston dynamics, and $(S_t^{2,i})_{1 \leq i \leq d'_1} = (S_t^i)_{d'_1+1 \leq i \leq d_1}$ are the corresponding variance swaps:

$$\begin{cases} dS_t^{1,i} = \sqrt{V_t^i} S_t^{1,i} dB_t^i, & S_0^{1,i} = s_0^i \\ dV_t^i = a^i(b^i - V_t^i)dt + \eta^i \sqrt{V_t^i} dW_t^i, & V_0^i = v_0^i \\ d\langle B^i, W^i \rangle_t = \rho^i \end{cases}$$

For the parameters:

$$d'_1 = 5, T = 1, a = 1_{\mathbb{R}^5}, b = 0.04 \times 1_{\mathbb{R}^5}, \eta = 2 \times 1_{\mathbb{R}^5}, \rho = -0.7 \times 1_{\mathbb{R}^5}, \alpha = 0.9$$

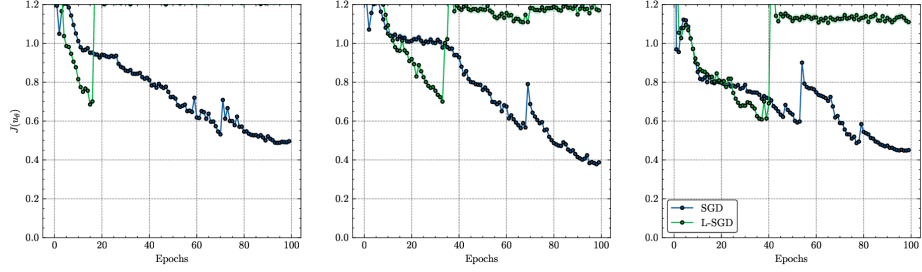
$$s_0 = K = 1_{\mathbb{R}^5}, v_0 = 0.1 \times 1_{\mathbb{R}^5}, c_{t_r} = 5e - 4 \times 1_{\mathbb{R}^5}.$$



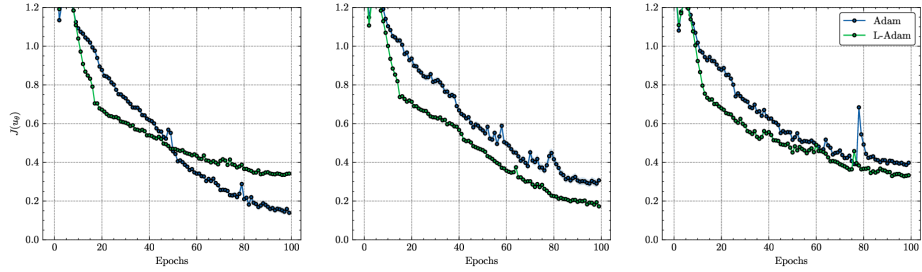
Example of a sample controlled trajectory for $N = 50$

In all our experiments, the control w was initialized to 0, and its optimum was reached using vanilla SGD with momentum 0.9, as adding Langevin noise to this single parameter wasn't necessary.

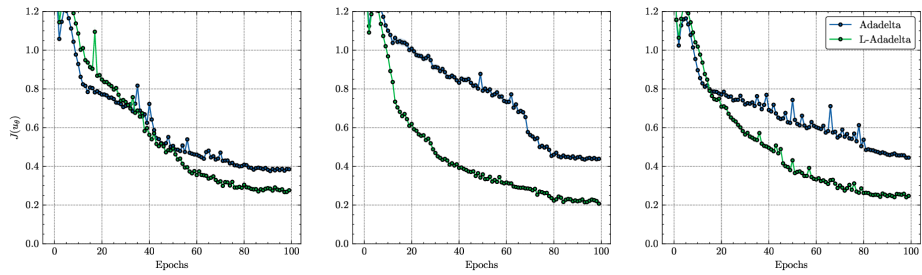
Single control In this problem, the control at time t_k is a function of $\log(S_{t_k}^1)$, V_{t_k} and $u_{t_{k-1}}$. We first train a single control $u_\theta : \mathbb{R}_+ \times \mathbb{R}^{10} \rightarrow \mathbb{R}^{10}$ using a two-layer feedforward network with each layer containing 32 hidden units and ReLU activation. The output is passed through a sigmoid function to ensure constraints on u . Each algorithm is trained for $N = 30, 50, 100$ respectively, and we plot the test loss for the vanilla algorithms and their Langevin counterparts.



Comparison of SGD and L-SGD with momentum = 0.9
 $N = 30, 50, 100$ over 100 epochs. The schedules are $\gamma_n = 3e - 2$ and $\sigma_n = 5e - 5$ between epochs 0 and 80, after which γ_n is divided by 10 and σ set to 0.



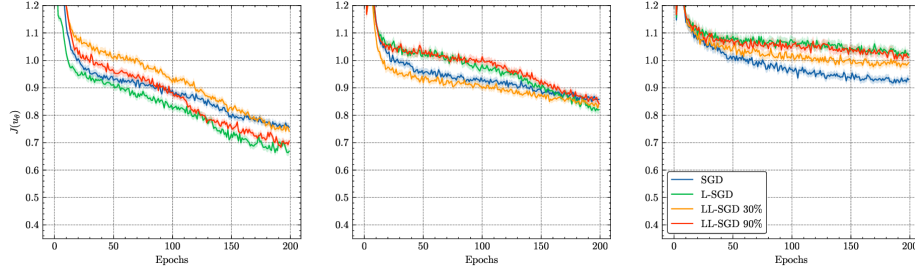
Comparison of Adam and L-Adam for $N = 30, 50, 100$ over 100 epochs. The schedules are $\gamma_n = 2e - 3$ and $\sigma_n = 2e - 4$ between epochs 0 and 80, after which γ_n is divided by 10 and σ set to 0.



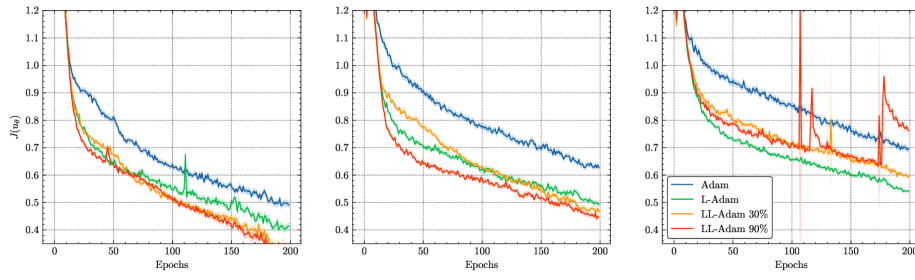
Comparison of Adadelta and L-Adadelta for $N = 30, 50, 100$ over 100 epochs. The schedules are $\gamma_n = 5e - 1$ and $\sigma_n = 5e - 3$ between epochs 0 and 80, after which γ_n is divided by 10 and σ set to 0.

The results in this problem are largely optimizer-dependent. Langevin dynamics provide a significant advantage only with Adadelta and Adagrad.

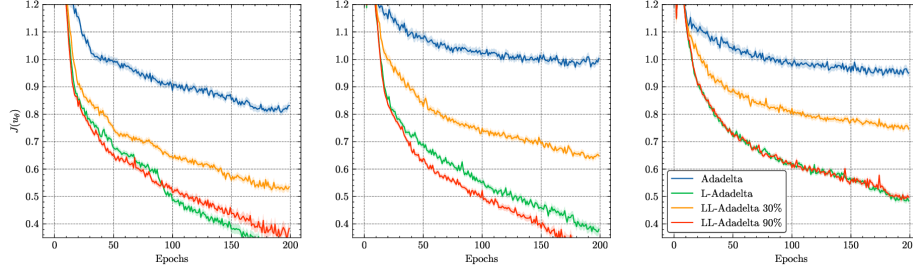
Multiple controls



Comparison of SGD, L-SGD, and LL-SGD with momentum = 0.9 for $N = 10, 20, 40$ over 200 epochs. The schedules are $\gamma_n = 5e - 2$ and $\sigma_n = 2e - 3$ between epochs 0 and 180, after which γ_n is divided by 10 and σ set to 0.



Comparison of Adam, L-Adam, and LL-Adam for $N = 10, 20, 40$ over 200 epochs. The schedules are $\gamma_n = 2e - 3$ and $\sigma_n = 2e - 4$ between epochs 0 and 180, after which γ_n is divided by 10 and σ set to 0.



Comparison of Adadelata, L-Adadelata, and LL-Adadelata for $N = 10, 20, 40$ over 200 epochs. The schedules are $\gamma_n = 5e - 1$ and $\sigma_n = 5e - 3$ between epochs 0 and 180, after which γ_n is divided by 10 and σ set to 0.

Langevin dynamics show a significant edge in performance, and the use of Langevin layers alone seems to balance computational cost and convergence speed without a clear advantage in using additional noisy layers.

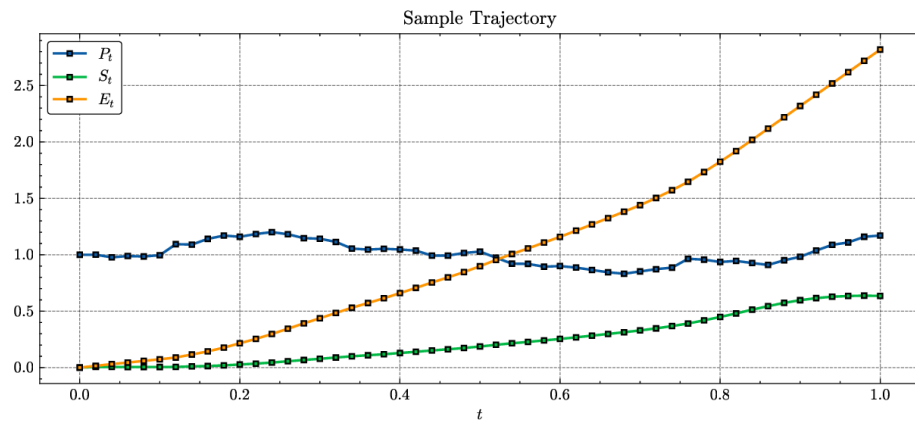
Oil Drilling

The oil driller balances extraction costs and storage in a volatile energy market. The oil price $P_t \in \mathbb{R}$ is assumed to follow a Black-Scholes process:

$$dP_t = \mu P_t dt + \eta P_t dW_t.$$

The control is given by $q_t = (q_t^v, q_t^s, q_t^{v,s}) \in \mathbb{R}^3$ with constraints on q_t^v (extracted oil immediately sold), q_t^s (stored), and $q_t^{v,s}$ (sold per time unit). The extraction and storage costs are given by $c_e(E_t)$ and $c_s(S_t)$:

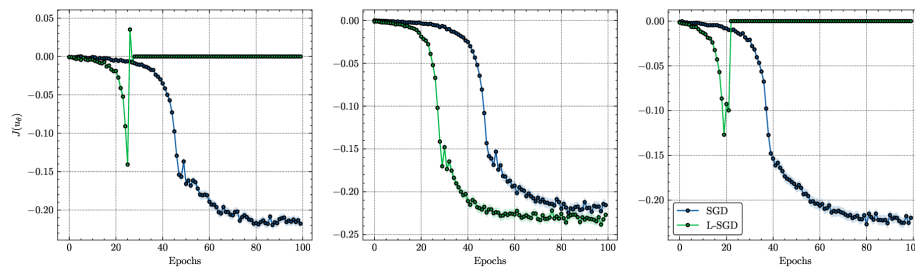
$$c_e(E_t) = \exp(\xi_e E_t), \quad c_s(S_t) = \exp(\xi_s S_t) - 1.$$



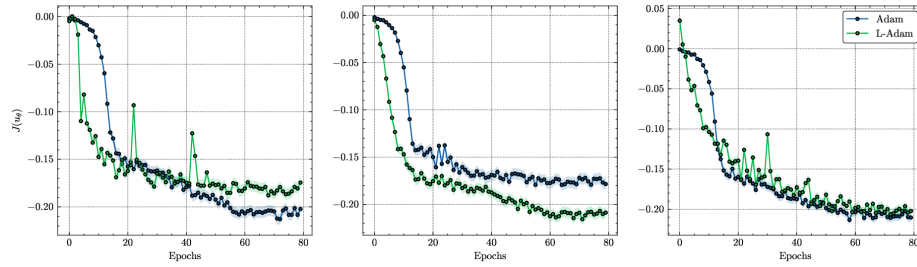
Example of a sample controlled trajectory for $N = 50$

The control q_t is given by a feedforward neural network with two hidden layers with 32 units and ReLU activation while the output layer has ReLU activations to meet constraints on q . Results are given for Langevin and non-Langevin algorithms.

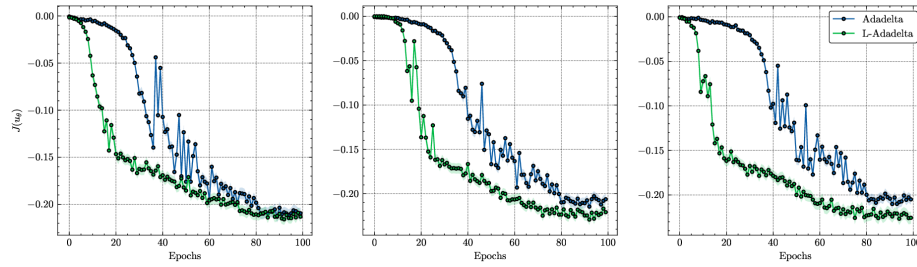
Single control



Comparison of SGD and L-SGD with momentum = 0.9 for $N = 30, 50, 100$ over 100 epochs. The schedules are $\gamma_n = 5e - 2$ and $\sigma_n = 1e - 3$ between epochs 0 and 80, after which γ_n is divided by 10 and σ set to 0.



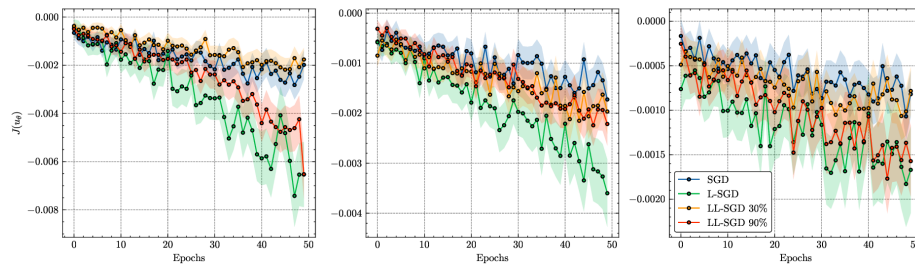
Comparison of Adam and L-Adam for $N = 30, 50, 100$ over 80 epochs. The schedules are $\gamma_n = 2e - 3$ and $\sigma_n = 1e - 3$ between epochs 0 and 60, after which γ_n is divided by 10 and σ set to 0.



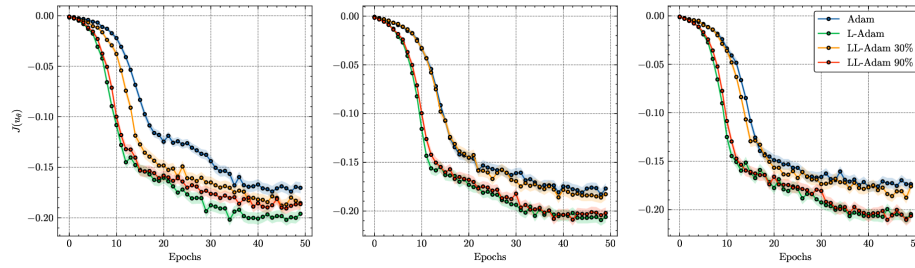
Comparison of Adadelta and L-Adadelta for $N = 30, 50, 100$ over 100 epochs. The schedules are $\gamma_n = 5e - 1$ and $\sigma_n = 5e - 3$ between epochs 0 and 80, after which γ_n is divided by 10 and σ set to 0.

The Langevin SGD appears to be unreliable, while Langevin Adadelta, RM-SProp, and Adagrad show better convergence than their traditional counterparts.

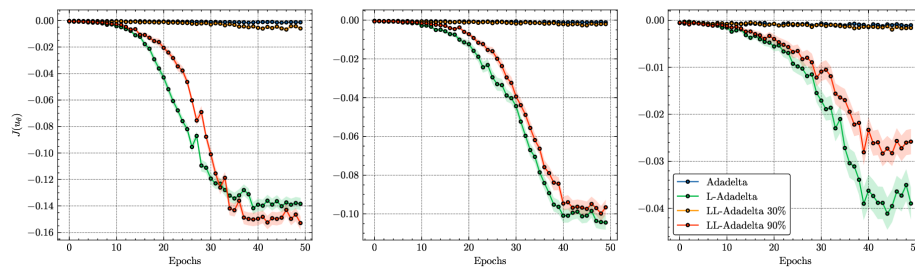
Multiple controls



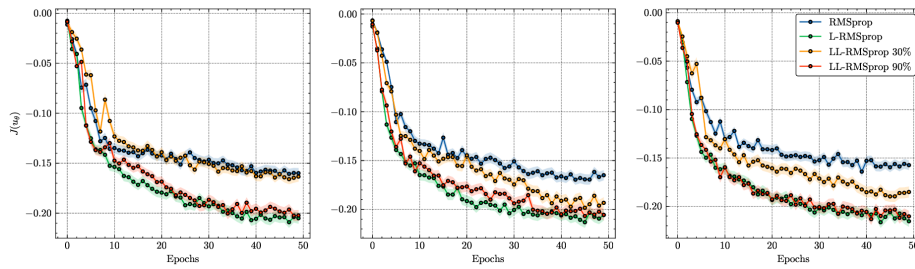
Comparison of SGD, L-SGD, and LL-SGD with momentum = 0.9 for $N = 10, 20, 40$ over 50 epochs. The schedules are $\gamma_n = 5e - 2$ and $\sigma_n = 2e - 3$ between epochs 0 and 40, after which γ_n is divided by 10 and σ set to 0.



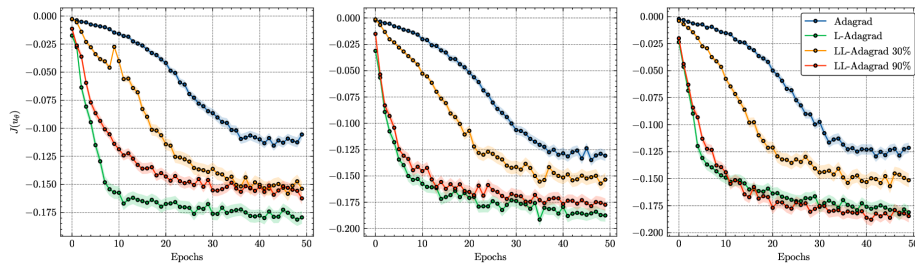
Comparison of Adam, L-Adam, and LL-Adam for $N = 10, 20, 40$ over 50 epochs. The schedules are $\gamma_n = 2e - 3$ and $\sigma_n = 2e - 4$ between epochs 0 and 40, after which γ_n is divided by 10 and σ set to 0.



Comparison of Adadelata, L-Adadelata, and LL-Adadelata for $N = 10, 20, 40$ over 50 epochs. The schedules are $\gamma_n = 5e - 1$ and $\sigma_n = 5e - 3$ between epochs 0 and 40, after which γ_n is divided by 10 and σ set to 0.



Comparison of RMSprop, L-RMSprop, and LL-RMSprop for $N = 10, 20, 40$ over 50 epochs. The schedules are $\gamma_n = 2e - 3$ and $\sigma_n = 2e - 3$ between epochs 0 and 40, after which γ_n is divided by 10 and σ set to 0.



Comparison of Adagrad, L-Adagrad, and LL-Adagrad for $N = 10, 20, 40$ over 50 epochs. The schedules are $\gamma_n = 5e - 3$ and $\sigma_n = 5e - 5$ between epochs 0 and 40, after which γ_n is divided by 10 and σ set to 0.

References

- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. 2015.
- T. Tieleman and G. E. Hinton. Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude. Coursera: Neural Networks for Machine Learning. 2012.
- Matthew D. Zeiler. ADADELTA: An Adaptive Learning Rate Method. 2012.
- Chandrasekaran Anirudh Bhardwaj. Adaptively Preconditioned Stochastic Gradient Langevin Dynamics. 2019.
- Pierre Bras. Langevin algorithms for very deep Neural Networks with applications to image classification. 2022.
- Pierre Bras and Gilles Pagès. Langevin algorithms for Markovian Neural Networks and Deep Stochastic control. 2023.

Mathieu Laurière, Gilles Pagès, and Olivier Pironneau. Performance of a Markovian neural network versus dynamic programming on a fishing control problem. 2021.